

# Chapter 4 Exam Notes: File Systems

## Modern Operating Systems Study Notes

### Contents

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Scope</b>                                       | <b>2</b> |
| <b>2</b> | <b>4.1 Files</b>                                   | <b>2</b> |
| 2.1      | Definition . . . . .                               | 2        |
| 2.2      | File Naming . . . . .                              | 2        |
| 2.3      | File Structure . . . . .                           | 2        |
| 2.4      | File Types . . . . .                               | 2        |
| 2.5      | How Executable Files Run . . . . .                 | 2        |
| 2.6      | File Access . . . . .                              | 3        |
| 2.7      | File Attributes . . . . .                          | 3        |
| 2.8      | Access Control . . . . .                           | 3        |
| 2.9      | File Operations . . . . .                          | 3        |
| <b>3</b> | <b>4.2 Directories</b>                             | <b>3</b> |
| 3.1      | Directory Definition . . . . .                     | 3        |
| 3.2      | Directory Structures . . . . .                     | 3        |
| 3.3      | Path Names . . . . .                               | 3        |
| 3.4      | Directory Operations . . . . .                     | 4        |
| <b>4</b> | <b>4.3 File-System Implementation</b>              | <b>4</b> |
| 4.1      | File-System Layout . . . . .                       | 4        |
| 4.2      | 4.3.2 Implementing Files . . . . .                 | 4        |
| 4.3      | I-node . . . . .                                   | 4        |
| 4.4      | Direct, Single-Indirect, Double-Indirect . . . . . | 4        |
| <b>5</b> | <b>Shared Files</b>                                | <b>5</b> |
| 5.1      | Hard Link . . . . .                                | 5        |

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| 5.2      | Symbolic Link . . . . .                                 | 5        |
| 5.3      | Shared File Problems . . . . .                          | 5        |
| <b>6</b> | <b>4.3.7 Virtual File System</b>                        | <b>5</b> |
| <b>7</b> | <b>Meaningful Figures and Steps</b>                     | <b>5</b> |
| 7.1      | Path Lookup Steps . . . . .                             | 5        |
| 7.2      | Reading a File by I-node . . . . .                      | 6        |
| <b>8</b> | <b>4.4 Management and Optimization: Small Questions</b> | <b>6</b> |
| 8.1      | Disk-Space Management . . . . .                         | 6        |
| 8.2      | Backups . . . . .                                       | 6        |
| 8.3      | Consistency . . . . .                                   | 6        |
| 8.4      | Performance . . . . .                                   | 6        |
| <b>9</b> | <b>Likely Questions with Answers</b>                    | <b>6</b> |

# 1 Scope

**Exam focus:** Study 4.1, 4.2, 4.3.2, i-nodes, shared files, 4.3.7 definitions, meaningful figures/steps, and smaller questions from 4.4. Skip 4.5 onward.

## 2 4.1 Files

### 2.1 Definition

A **file** is a named collection of information stored on secondary storage. It is the OS abstraction for persistent data.

### 2.2 File Naming

Names are for users and programs. Rules depend on the OS: case sensitivity, allowed characters, length, and extension conventions.

### 2.3 File Structure

- **Byte sequence:** OS treats file as uninterpreted bytes. Common in UNIX.
- **Record sequence:** file is a sequence of fixed/variable records.
- **Tree:** records organized by keys, used in some older systems.

### 2.4 File Types

- **Regular files:** user data or programs.
- **Directories:** system files mapping names to files.
- **Character special files:** serial devices such as terminals.
- **Block special files:** block devices such as disks.

### 2.5 How Executable Files Run

1. Shell or parent process requests execution.
2. OS opens the executable and checks permissions/type.
3. Loader reads executable header.
4. Text, data, bss, stack, heap, and libraries are mapped/loaded.
5. Initial registers and stack are set.
6. Process begins at entry point.

## 2.6 File Access

- **Sequential access:** read bytes/records in order.
- **Random access:** seek to arbitrary offset and read/write.

## 2.7 File Attributes

Common attributes: size, owner, permissions, timestamps, type, flags, location metadata, and link count.

## 2.8 Access Control

Access control decides who can read, write, execute, delete, or change metadata.

- UNIX-style: owner/group/others permission bits.
- ACL: list of users/groups and allowed operations.

## 2.9 File Operations

Create, delete, open, close, read, write, append, seek, get attributes, set attributes, rename.

# 3 4.2 Directories

## 3.1 Directory Definition

A directory maps file names to file metadata or metadata pointers.

## 3.2 Directory Structures

- Single-level directory: simple but name conflicts.
- Hierarchical directory: tree structure with paths.

## 3.3 Path Names

- **Absolute path:** starts at root.
- **Relative path:** starts at current directory.
- **.**: current directory.
- **..**: parent directory.

### 3.4 Directory Operations

Create, delete, open, close, read, rename, link, unlink.

## 4 4.3 File-System Implementation

### 4.1 File-System Layout

Typical disk layout includes boot block, superblock, free-space management data, i-nodes or meta-data, root directory, directories, and file data blocks.

### 4.2 4.3.2 Implementing Files

**Contiguous allocation** Each file occupies consecutive blocks. Fast sequential and random access, but external fragmentation and growth problems.

**Linked-list allocation** Each block points to the next. No external fragmentation, but random access is slow.

**FAT** Pointers are kept in a File Allocation Table in memory. Faster traversal than on-disk linked list, but large FAT can consume memory.

**I-node allocation** Each file has an index node listing its blocks or indirect block pointers.

### 4.3 I-node

An **i-node** is a per-file metadata structure. It stores attributes and pointers to data blocks.

Typical i-node fields:

- file type and permissions,
- owner and group,
- size,
- timestamps,
- link count,
- direct and indirect block pointers.

### 4.4 Direct, Single-Indirect, Double-Indirect

- Direct pointer points directly to a data block.
- Single-indirect pointer points to a block full of data-block addresses.
- Double-indirect pointer points to a block of single-indirect block addresses.
- Triple-indirect extends this one more level for very large files.

## 5 Shared Files

### 5.1 Hard Link

Two directory entries point to the same i-node. The file remains until link count becomes zero and no process has it open.

### 5.2 Symbolic Link

A file containing a path name to another file. Can cross file-system boundaries but may dangle if the target is removed.

### 5.3 Shared File Problems

- Correct link count maintenance.
- Avoiding duplicated block ownership.
- Handling deletion while another user still links or opens the file.
- Consistency after crashes.

## 6 4.3.7 Virtual File System

**VFS** is an abstraction layer that lets multiple file-system implementations provide a common interface.

- User programs call common file operations.
- VFS dispatches operations to the correct concrete file system.
- Supports local, network, and special file systems through one interface.

## 7 Meaningful Figures and Steps

### 7.1 Path Lookup Steps

For `/usr/bin/x`:

1. Start at root directory.
2. Find entry `usr`.
3. Read `usr` directory.
4. Find entry `bin`.

5. Read `bin` directory.
6. Find entry `x`.

## 7.2 Reading a File by I-node

1. Open path and locate directory entry.
2. Load i-node.
3. Use file offset to compute logical block number.
4. Use direct/indirect pointers to find disk block.
5. Read block into cache.
6. Copy requested bytes to user buffer.

## 8 4.4 Management and Optimization: Small Questions

### 8.1 Disk-Space Management

- Block size tradeoff: small blocks reduce internal fragmentation; large blocks improve throughput and reduce metadata.
- Free blocks can be tracked by bitmaps or linked lists.

### 8.2 Backups

Backups protect against disaster and user/admin mistakes. Incremental backups save changes since last backup.

### 8.3 Consistency

Crashes can leave metadata inconsistent. Checkers and journaling reduce repair cost.

### 8.4 Performance

Buffer cache, block read-ahead, reducing disk seeks, and careful placement improve file-system speed.

## 9 Likely Questions with Answers

| Question                                          |
|---------------------------------------------------|
| Q1. Define file, directory, path, i-node, and VFS |

### Answer

A file is a named collection of persistent information. A directory maps names to files or other directories. A path is a sequence of directory names leading to a file. An i-node is an on-disk data structure storing file metadata and pointers to file blocks. VFS, or Virtual File System, is an OS layer that gives programs one common file-system interface while allowing different real file systems underneath.

### Question

**Q2. Explain file structures and file types**

### Answer

Common file structures are byte sequence, record sequence, and tree. In a byte sequence, the OS treats the file as uninterpreted bytes. In a record sequence, the file is made of records. In a tree, records are organized by keys. Common file types include regular files, directories, character special files, and block special files.

### Question

**Q3. Explain executable loading steps**

### Answer

A shell or parent process requests execution. The OS opens the executable, checks type and permissions, reads the executable header, maps or loads code/data/bss, prepares heap and stack, loads required libraries if needed, sets the initial registers and entry point, and starts execution.

### Question

**Q4. Compare contiguous, linked, FAT, and i-node allocation**

### Answer

Contiguous allocation stores a file in consecutive blocks, giving fast sequential and random access, but it suffers from external fragmentation and file growth problems. Linked allocation stores each block with a pointer to the next, making growth easy but random access slow. FAT keeps the linked-list pointers in a table in memory, improving traversal but requiring memory for the table. I-node allocation stores block pointers in an i-node, supports random access well, and can grow through direct and indirect blocks.

### Question

#### Q5. Hard link vs symbolic link

### Answer

A hard link is another directory entry pointing to the same underlying file/i-node. The file remains until all hard links are removed. A symbolic link is a special file containing a pathname to another file. It can cross file-system boundaries, but it can become dangling if the target is removed.

### Question

#### Q6. Explain shared-file issues

### Answer

Shared files create consistency and ownership issues. If one user changes or deletes a shared file, other users' directory entries may be affected. Hard links require link counts so the system knows when the actual file can be deleted. Symbolic links avoid some hard-link restrictions but can break if the target pathname disappears.

### Question

#### Q7. Describe path lookup or file read steps

### Answer

For path lookup, the OS starts from the root directory for an absolute path or current directory for a relative path. It searches each directory component, finds the corresponding metadata, checks permissions, and continues until the final file is found. For a file read, the OS checks the file descriptor and permissions, uses metadata such as the i-node to locate disk blocks, reads needed blocks through the buffer cache/device driver, copies data to the user buffer, and updates the file offset.